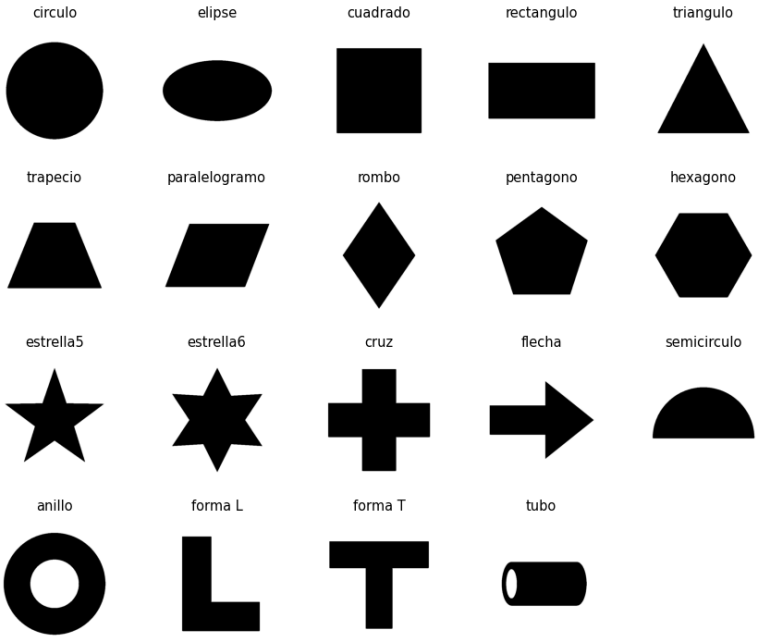
Detector de figuras geométricas para manufactura

Guía completa: qué hace, cómo funciona y cómo publicarlo

Aplicación web que corre 100 % en GitHub Pages. Recibe una imagen de 512 × 512 px (fondo blanco, figura negra) y responde cuál de las **19 figuras** predefinidas contiene, o «figura no reconocida». Todo el procesamiento ocurre en el navegador del usuario con Python (Pyodide); ninguna imagen se sube a un servidor.



Las 19 figuras que reconoce el sistema.

Contenido: 1 Visión general · 2 Flujo paso a paso · 3 Las figuras · 4 Preprocesado · 5 Características · 6 Grafo representativo · 7 Búsqueda y decisión · 8 Algoritmo genético · 9 Resultados · 10 Cómo funciona la página web · 11 Publicar en GitHub · 12 Mantenimiento · 13 Limitaciones

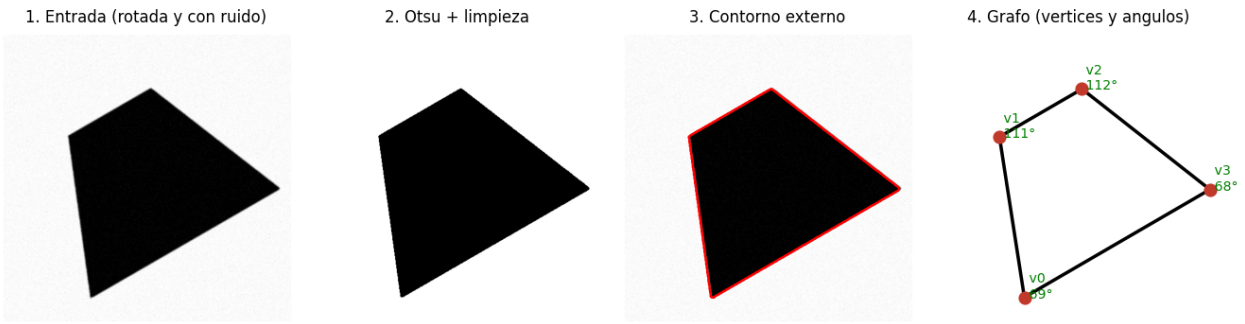
1. Visión general

El sistema no usa aprendizaje profundo. Combina técnicas clásicas, cada una con un papel claro:

Técnica	Para qué se usa
Visión por computador	Convertir la imagen en un contorno limpio y medir sus propiedades (vértices, solidez, huecos…)
Grafo representativo	Describir la estructura de la figura (ángulos y longitudes de lados) y verificar la decisión
Problemas de búsqueda	Encontrar la plantilla más cercana (1-NN), recorrer el grafo (BFS/DFS) y alinear grafos (A*)
Algoritmo genético	Encontrar automáticamente cuánto pesa cada característica y cuál es el umbral de rechazo
Pyodide	Ejecutar Python dentro del navegador, para que todo viva en GitHub Pages

Regla de oro: el sistema solo promete funcionar con imágenes del mismo tamaño y estilo que las plantillas (512 × 512, fondo blanco, figura negra). Ante cualquier otra forma responde «figura no reconocida».

2. Flujo paso a paso



Ejemplo real: un trapecio rotado 30° y con ruido atraviesa las etapas del detector.

Etapa	Qué ocurre	Dónde
1. Carga	El usuario sube o arrastra una imagen. El navegador la dibuja en un canvas de 512 × 512 y la pasa a gris. Si el tamaño no es 512 × 512 avisa.	index.html
2. Preprocesado	Filtro gaussiano, umbral de Otsu, limpieza morfológica, componente mayor, contorno externo y conteo de huecos.	detector.py
3. Características	Vector de 10 números invariantes a rotación, posición y escala.	detector.py
4. Grafo	El contorno simplificado se convierte en un grafo de vértices (con ángulo) y aristas (con longitud).	detector.py
5. Búsqueda	Distancia ponderada a cada una de las 19 plantillas; se elige la menor.	detector.py
6. Decisión	Si la distancia supera el umbral τ: «figura no reconocida». Si no, se muestra la figura y la confianza. El grafo verifica y puede bajar la confianza.	detector.py

3. Las 19 figuras

Son: círculo, elipse, cuadrado, rectángulo, triángulo, trapecio, paralelogramo, rombo, pentágono, hexágono, estrella de 5 puntas, estrella de 6 puntas, cruz, flecha, semicírculo, anillo, forma L, forma T y

tubo. Cada una está en `figuras/` como PNG de 512 × 512. **El octágono se excluyó** porque en las pruebas se confundía sistemáticamente con el hexágono; el plan original tenía 20 figuras.

Figura	Vértices	Solidez	Aspecto	Huecos	CV radial
circulo	8	0.997	1.00	0	0.001
elipse	8	0.996	1.80	0	0.194
cuadrado	4	1.000	1.00	0	0.110
rectangulo	4	1.000	1.91	0	0.240
triangulo	3	0.995	1.02	0	0.230
trapecio	4	0.997	1.44	0	0.165
paralelogramo	4	0.997	1.64	0	0.186
rombo	4	0.994	1.46	0	0.182
pentagono	6	0.996	1.05	0	0.066
hexagono	6	0.996	1.15	0	0.044
estrella5	10	0.500	1.05	0	0.260
estrella6	12	0.629	1.15	0	0.179
cruz	12	0.722	1.00	0	0.222
flecha	7	0.728	1.17	0	0.303
semicirculo	5	0.997	2.00	0	0.238
anillo	8	0.997	1.00	1	0.001
forma L	6	0.728	1.22	0	0.394
forma T	8	0.666	1.14	0	0.352
tubo	6	0.998	1.94	1	0.228

Estos son valores medidos por el propio sistema sobre las imágenes modelo (están en `plantillas.json`). Varias figuras comparten el mismo número de vértices (por ejemplo círculo, elipse, anillo y tubo dan 8), por eso hace falta combinar varias características.

4. Preprocesado de la imagen

- **Gris y suavizado.** Filtro gaussiano ($\sigma = 1$) para reducir ruido antes de umbralizar.
- **Umbral de Otsu** (Otsu, 1979). Elige el umbral t que maximiza la varianza entre las dos clases de píxeles (fondo y figura). No hay que fijarlo a mano y se adapta a cambios de brillo. Si la imagen resulta invertida (figura clara sobre fondo oscuro), se invierte automáticamente.
- **Limpieza morfológica.** Cierre y apertura con un disco de radio 1 eliminan puntos sueltos y huecos diminutos.
- **Componente mayor.** Se conserva la región conectada más grande, que es la figura.
- **Contorno externo.** Con `find_contours` de scikit-image se toma el contorno más largo.
- **Huecos.** Son las regiones de fondo que no tocan el borde de la imagen (mayores de 30 píxeles). Distinguen anillo y tubo de círculo y rectángulo.

5. Extracción de características

Cada figura se describe con un vector de 10 valores. Todos son invariantes a rotación, posición y escala:

Característica	Definición	Para qué sirve
Vértices	Puntos del contorno tras simplificar con Douglas-Peucker, con ε = 2 % del perímetro	Separa triángulo, cuadriláteros, pentágono, hexágono, cruz, flecha, estrellas
Solidez	Área del contorno / área de su envolvente convexa	Figuras convexas ≈ 1; cóncavas (estrellas, L, cruz) valen menos
Aspecto	Lado mayor / lado menor del rectángulo mínimo rotado	Cuadrado vs rectángulo, círculo vs elipse
Huecos	Número de regiones internas	Anillo y tubo
CV radial	Desviación típica / media de la distancia del centroide al contorno	Círculo (≈ 0) frente a polígonos regulares
log CV radial	$\log_{10}$(CV radial)	Amplía las diferencias entre valores muy pequeños
Hu1 a Hu4	Primeros 4 momentos invariantes de Hu (1962), transformados con $-\text{signo}\cdot\log_{10}\lvert h\rvert$	Descripción global de la forma

Douglas-Peucker (1973). Une el primer y el último punto con una recta; si el punto más alejado está a más de ε, lo conserva y repite en cada mitad; si no, descarta los intermedios.

Detalle importante. Los momentos de Hu de figuras muy simétricas (círculo, cuadrado) valen casi cero, y su logaritmo se dispara por diferencias numéricas diminutas. Por eso el valor se recorta a un mínimo de 10^{-7} antes del logaritmo. Sin este recorte el octágono rotado se confundía con el hexágono en todas las pruebas.

6. Grafo representativo

Un grafo G = (V, E) es un conjunto de vértices unidos por aristas. El **grafo representativo** de una figura es el que resume su estructura geométrica:

- **Vértices:** los puntos de Douglas-Peucker. Cada uno guarda su **ángulo interior** (mayor de 180° si es cóncavo).
- **Aristas:** los segmentos entre vértices consecutivos. Cada una guarda su **longitud normalizada** (longitud / perímetro), independiente de la escala.
- **Estructura:** un ciclo de n vértices. Se implementa con NetworkX.

En el ejemplo de la sección 2, el trapecio da un grafo de 4 vértices con ángulos de aproximadamente 69°, 111°, 112° y 68°, y la suma es 360°, como corresponde a un cuadrilátero.

Cómo se comparan dos grafos. Una figura rotada puede producir el grafo con otro vértice de inicio o recorrido en sentido contrario. Por eso la distancia es el mínimo sobre todos los desplazamientos cíclicos s y ambos sentidos:

$$d(G1, G2) = \min_{s,\ \text{sentido}} \Sigma_i \left(\lvert \text{áng1}[i] - \text{áng2}[i+s] \rvert / 180 + \lvert \text{long1}[i] - \text{long2}[i+s] \rvert \right)$$

Si los grafos tienen distinto número de vértices la distancia es infinita. El grafo resuelve parejas como trapecio y paralelogramo, o rombo y rectángulo, que comparten número de vértices pero difieren en ángulos y lados. En el sistema final **actúa como verificación**: si la plantilla propuesta por las características no coincide con el grafo, la confianza se multiplica por 0.7. Para figuras curvas el grafo es poco informativo y decide el vector de características.

7. Problemas de búsqueda y decisión

Un problema de búsqueda se define por un estado inicial, acciones, costos y una prueba de meta. Aparecen tres:

Problema	Definición	Algoritmo
Plantilla más cercana	Estado: una de las 19 plantillas. Acción: evaluar la siguiente. Costo: distancia ponderada. Meta: la de menor distancia si es menor que el umbral.	Vecino más cercano (1-NN), exhaustivo y óptimo: 19 comparaciones
Recorrido del grafo	Obtener el orden de los vértices y detectar componentes.	BFS (cola, por niveles) y DFS (pila, en profundidad); O(n + m). La página muestra ambos órdenes
Alineación de grafos	Hallar el desplazamiento y sentido que alinean dos grafos. Estado: asignación parcial; costo g(n): error acumulado.	A* (Hart, Nilsson y Raphael, 1968). Con n ≤ 12 se prueban todos los desplazamientos, equivalente y suficiente

Distancia al vector de características:

$$d = \sqrt{\Sigma_k \; w_k \cdot ((x_k - p_k) / s_k)^2}$$

donde x es el vector de la imagen, p el de la plantilla, s_k la desviación típica de cada característica entre las plantillas (normaliza escalas distintas) y w_k los pesos que aprende el algoritmo genético.

Regla de decisión y rechazo:

```
d_min = distancia a la plantilla más cercana
si d_min > τ: «figura no reconocida»
si no: figura = plantilla más cercana, confianza = 1 − d_min / τ
(si el grafo no verifica, confianza × 0.7)
```

El umbral aprendido es τ = 0.360. Debe ser lo bastante grande para aceptar la misma figura rotada, escalada o con ruido, y lo bastante pequeño para rechazar formas ajenas.

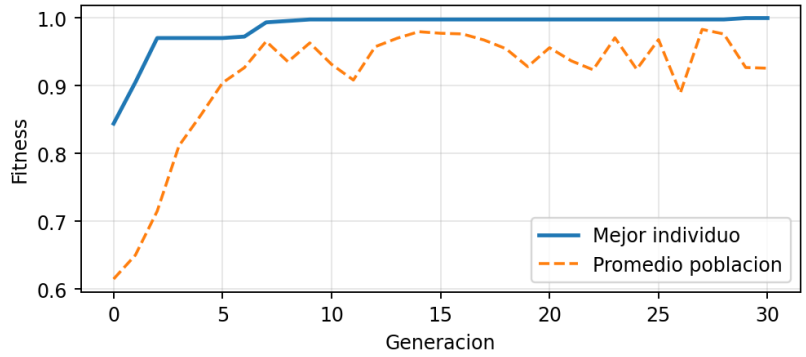
8. Algoritmo genético

Un algoritmo genético (Holland, 1975) hace evolucionar una población de candidatos mediante selección, cruce y mutación. Aquí decide **cuánto pesa cada característica** y **cuál es el umbral de rechazo**, algo difícil de ajustar a mano.

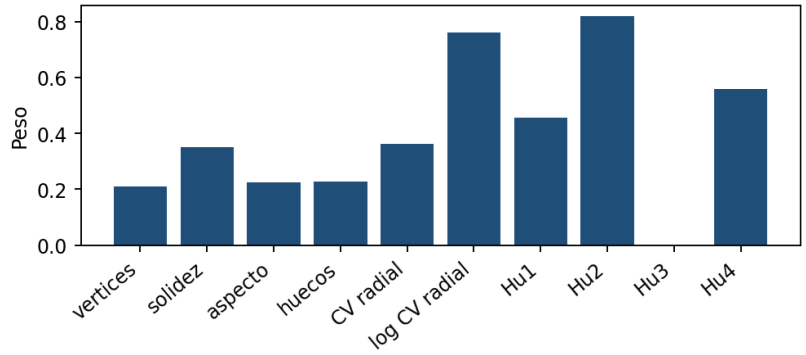
Componente	Diseño
Cromosoma	10 pesos w en [0, 1] + 1 gen para el umbral (se multiplica por 8)
Población	40 individuos aleatorios
Aptitud (fitness)	0.6 × (aciertos aceptados en entrenamiento) + 0.4 × (figuras desconocidas rechazadas)
Selección	Torneo de 3 individuos
Cruce	Uniforme: cada gen viene de uno de los dos padres con probabilidad 0.5
Mutación	Probabilidad 0.1 por gen; ruido gaussiano σ = 0.15; valores limitados a [0, 1]
Elitismo	El mejor individuo pasa intacto a la siguiente generación
Parada	30 generaciones

```
población = aleatoria(40)
repetir 30 veces:
  aptitud = [fitness(ind) para ind en población]
  nueva = [mejor(población)] # elitismo
  mientras len(nueva) < 40:
    a = torneo(población, 3); b = torneo(población, 3)
```

```
    hijo = mutar(cruce_uniforme(a, b))
    nueva.append(hijo)
  población = nueva
devolver mejor(población)
```



Convergencia real del entrenamiento: el mejor individuo mejora rápido y se estabiliza.



Pesos aprendidos. Un peso cercano a 0 significa que esa característica casi no aporta (aquí, Hu3).

El entrenamiento se hace una sola vez, fuera del navegador, con `entrenar.py`. Los pesos y el umbral resultantes se guardan en `plantillas.json`, y la página solo los usa.

9. Resultados

Se midieron sobre imágenes sintéticas generadas a partir de las 19 plantillas con rotación aleatoria de 0° a 360°, escala de 0.6 a 1.0, desplazamiento de hasta 40 px, ruido gaussiano y desenfoque. Entrenamiento: 285 imágenes; prueba: 570 imágenes (30 por figura); figuras ajenas: 60 imágenes (heptágono, triángulo irregular, hexágono achatado, pentágono irregular).

Configuración	Resultado
Pesos iguales, sin genético	99.8 %
Con genético, sin umbral (clasificación pura)	99.8 %
Con genético y umbral de rechazo	99.1 %
Rechazo de figuras desconocidas	100.0 %

Cómo leerlos con cuidado:

- Son imágenes sintéticas con el mismo estilo que las plantillas, justo el escenario del proyecto. **No garantizan el mismo resultado con fotografías reales.**

- Las figuras ajenas de entrenamiento y de prueba son los mismos 4 tipos, por lo que el 100 % de rechazo es optimista. Con formas ajenas distintas puede bajar.
- Con 19 figuras limpias, los pesos iguales ya aciertan casi todo; la mejora clave vino de corregir la inestabilidad de Hu. El genético aporta sobre todo el **umbral de rechazo** y el ajuste de pesos.
- Se superan los criterios de aceptación del plan (≥ 95 % de acierto y ≥ 90 % de rechazo) en las pruebas sintéticas. Conviene volver a medir con las imágenes reales que se vayan a usar.

10. Cómo funciona la página web

GitHub Pages solo sirve archivos estáticos: no ejecuta Python en un servidor. Para cumplir «todo en GitHub» se usa **Pyodide**, una versión de Python compilada a WebAssembly que corre dentro del navegador.

Archivo	Contenido
index.html	Interfaz, carga de Pyodide, subida de imagen y cuatro pestañas: Detectar, Algoritmos, Biblioteca de figuras y Genético
detector.py	Preprocesado, características, grafo, BFS/DFS, búsqueda y clasificación
genetico.py	Algoritmo genético en NumPy (usado por entrenar.py)
plantillas.json	Características y grafos de las 19 plantillas, pesos y umbral optimizados, curva de convergencia y resultados
figuras/	Las 19 imágenes modelo (PNG 512 × 512)
entrenar.py	Script opcional, local: regenera plantillas.json
README.md	Resumen y pasos de publicación

Qué pasa al abrir la página:

- El navegador descarga Pyodide desde un CDN (jsDelivr) y los paquetes NumPy, SciPy, scikit-image y NetworkX. La primera carga puede tardar unos segundos; después queda en caché.
- La página lee `detector.py`, `genetico.py` y `plantillas.json` y los escribe en el sistema de archivos virtual de Pyodide.
- Al subir una imagen, JavaScript la dibuja en un canvas de 512 × 512, la pasa a escala de grises y envía los píxeles a Python.
- Python ejecuta el detector y devuelve un resultado en JSON; JavaScript dibuja el contorno (rojo) y los vértices (azul) y muestra la figura detectada, la confianza, las 3 plantillas más cercanas, las 10 características, los ángulos y longitudes del grafo, y los recorridos BFS y DFS.
- Tras cargar los paquetes, el procesamiento es local. La página solo hace peticiones de red para descargar Pyodide, los paquetes y sus propios archivos. La imagen del usuario no sale del navegador.

Usar la página: arrastra una imagen o haz clic en el recuadro para elegirla. También puedes ir a «Biblioteca de figuras» y hacer clic en cualquier figura para probarla. Si la figura no coincide con ninguna, verás «Figura no reconocida» junto con la plantilla más cercana y la distancia.

11. Publicar en GitHub

Paso	Acción
1	Crea un repositorio nuevo en GitHub (público).
2	Sube todo el contenido del zip a la raíz del repositorio, incluida la carpeta figuras/. Puedes arrastrar los archivos desde la interfaz web de GitHub o usar git.

Paso	Acción
3	Ve a Settings → Pages. En «Build and deployment» elige Source: Deploy from a branch, Branch: main, carpeta / (root). Guarda.
4	Espera unos minutos. GitHub muestra la URL https://USUARIO.github.io/REPOSITORIO/
5	Abre la URL y prueba con una figura de la biblioteca.

Si algo falla:

- **Se queda en «Cargando…»:** revisa la conexión (se necesita internet para descargar Pyodide) y abre la consola del navegador (F12) para ver el error.
- **Error de archivo no encontrado:** comprueba que index.html, detector.py, genetico.py, plantillas.json y la carpeta figuras/ estén en la raíz, no dentro de otra carpeta.
- **No se abre la página:** Pages tarda unos minutos tras activarse; confirma en la pestaña Actions que el despliegue terminó.
- **No funciona abriendo index.html con doble clic:** el navegador bloquea la lectura de archivos locales. Debe verse desde la URL de GitHub Pages o con un servidor local (`python -m http.server`).

Aviso: el código se probó en Python (todas las figuras rotadas se reconocieron), pero no se pudo probar dentro de un navegador real con Pyodide. Si hay un error al desplegar, el mensaje de la consola indica la causa.

12. Mantenimiento y ampliación

- **Reentrenar:** instala `numpy scipy scikit-image networkx pillow` y ejecuta `python entrenar.py` en la carpeta del proyecto. Regenera plantillas.json con nuevos pesos y umbral.
- **Agregar una figura:** añade un PNG de 512 × 512 a figuras/ con nombre `NN_nombre.png`, reentrena y actualiza en index.html la función que asocia el índice con el número de archivo. Revisa que no se confunda con otra figura.
- **Quitar una figura:** elimina su PNG y reentrena (así se retiró el octágono).
- **Cambiar el rigor del rechazo:** el umbral τ está en plantillas.json; subirlo acepta más variaciones y baja el rechazo.

13. Limitaciones

Riesgo	Detalle
Primera carga lenta	Pyodide y los paquetes pesan varios MB; luego quedan en caché del navegador
Otro estilo o tamaño de imagen	Solo se promete 512 × 512, fondo blanco y figura negra; otro tamaño se reescala y avisa
Fotos reales, sombras o varias piezas	Fuera de alcance; requeriría un clasificador entrenado y detección por instancias
Figuras ajenas muy parecidas a una plantilla	Pueden ser aceptadas (por ejemplo, un cuadrilátero casi idéntico a un trapecio)
Pruebas sintéticas	Los porcentajes deben volver a medirse con las imágenes reales que se vayan a usar
Dependencia de internet	Pyodide se descarga desde un CDN en cada visita nueva

Referencias

Otsu, N. (1979). A threshold selection method from gray-level histograms. IEEE Trans. on Systems, Man, and Cybernetics.

Douglas, D. y Peucker, T. (1973). Algorithms for the reduction of the number of points required to represent a digitized line. The Canadian Cartographer.

Hu, M.-K. (1962). Visual pattern recognition by moment invariants. IRE Trans. on Information Theory.

Hart, P., Nilsson, N. y Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. IEEE Trans. on Systems Science and Cybernetics.

Holland, J. (1975). Adaptation in Natural and Artificial Systems. University of Michigan Press.

Documentación de Pyodide, scikit-image, SciPy, NumPy y NetworkX.